Examen — Codage et traitement du signal

Clones fonctionnels des utilitaires de compression bzip2, lzma et 7-zip

Auteur : Steph

Date limite de remise : samedi 24 octobre 2026, 23h59

Langage : Python 3 (bibliothèque standard uniquement ; les algorithmes sont réimplémentés, `zlib.crc32` sert seulement de somme de contrôle).

Dépôt GitHub : (coller ici le lien du dépôt)

1. Algorithmes implémentés

bzip2-clone — même chaîne de traitement que bzip2 : pré-codage RLE, transformée de Burrows-Wheeler (tri des rotations par doublement de préfixes), Move-To-Front, codage des plages de zéros (RUNA/RUNB), puis codage de Huffman canonique par blocs de 100 à 900 Ko.

lzma-clone — compression par dictionnaire LZ77 (chaînes de hachage, fenêtre jusqu'à 64 Mo) suivie d'un codeur par intervalles binaire adaptatif (probabilités sur 11 bits). Littéraux contextuels, longueurs 2 à 273, distances par emplacements + bits directs, répétition de la dernière distance (rep0).

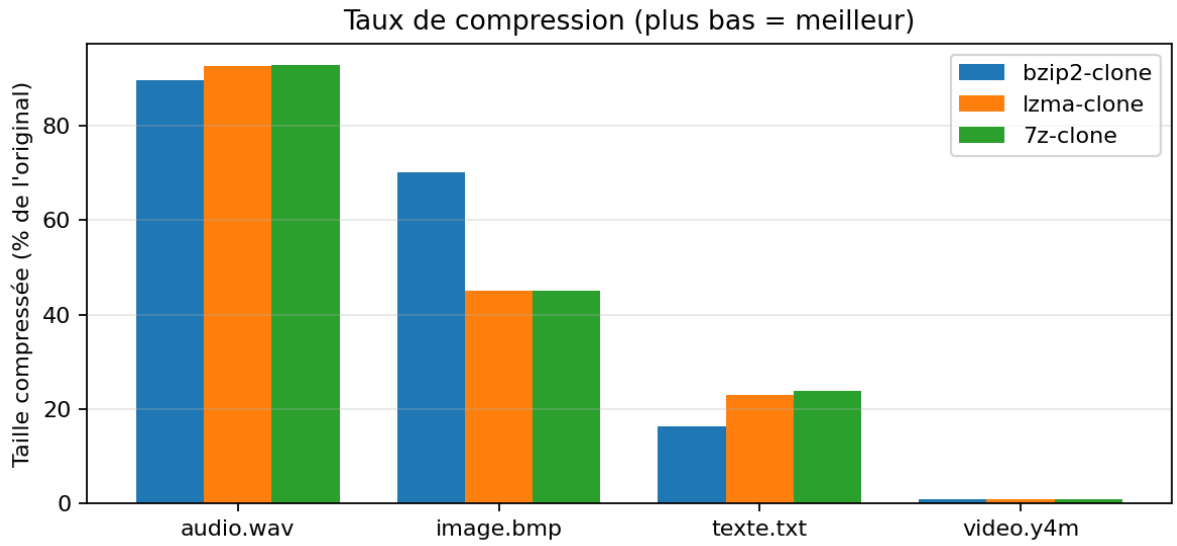
7z-clone — archiveur « solide » : tous les fichiers sont concaténés puis compressés par le moteur lzma-clone ; l'entête (noms, tailles, dates, CRC32) est lui-même compressé. Commandes `a` (créer), `x` (extraire), `l` (lister), `t` (tester), avec protection contre la traversée de répertoires.

Remarque : les formats de fichier produits (.bzc, .lzc, .7zc) sont propres à ce projet : ils ne sont pas lisibles par les outils officiels, mais l'aller-retour est vérifié par SHA-256.

2. Résultats comparatifs

Quatre types de fichiers ont été testés. Les colonnes « réf. » donnent, à titre de comparaison, la taille obtenue par les bibliothèques officielles bz2 (niveau 9) et lzma/xz (préréglage 6).

Fichier	Type	Original (octets)	bzip2-clone	lzma-clone	7z-clone	bzip2 réf.	xz réf.	Aller-retour
audio.wav	audio	264 644	237 596 (89.8 %)	245 576 (92.8 %)	245 697 (92.8 %)	220 664	240 304	identique
image.bmp	image	230 454	161 513 (70.1 %)	103 851 (45.1 %)	103 973 (45.1 %)	154 549	56 504	identique
texte.txt	texte	310 768	50 838 (16.4 %)	71 545 (23.0 %)	73 848 (23.8 %)	43 284	58 392	identique
video.y4m	vidéo	576 163	4 641 (0.8 %)	4 898 (0.9 %)	5 105 (0.9 %)	3 505	2 228	identique



3.1 Fichier texte : texte.txt

Le texte est très redondant (mots répétés) : tous les clones obtiennent un très bon taux. bzip2-clone fait mieux que lzma-clone ici, comme pour l'original, car BWT + Huffman exploitent bien les contextes statistiques d'un texte.

Taille originale : **310 768 o** — bzip2-clone : **50 838 o** (16.4 %) — lzma-clone : **71 545 o** (23.0 %) — 7z-clone : **73 848 o** (23.8 %).

Session de terminal :

```
$ python clonecompress.py bzip2 -k -f -v -n 3 -o resultats/texte.txt.bzc echantillons/texte.txt
echantillons/texte.txt -> resultats/texte.txt.bzc : 303.48 Ko -> 49.65 Ko (16.36 % de l'original, gain 83.64 %)
    en 0.89 s
$ python clonecompress.py lzma -k -f -v -n 6 -o resultats/texte.txt.lzc echantillons/texte.txt
echantillons/texte.txt -> resultats/texte.txt.lzc : 303.48 Ko -> 69.87 Ko (23.02 % de l'original, gain 76.98 %)
    en 0.43 s
$ python clonecompress.py 7z a resultats/texte.txt.7zc echantillons/texte.txt -n 5
Archive resultats/texte.txt.7zc créée : 1 fichier(s), 303.48 Ko -> 72.12 Ko (23.76 % de l'original)
Durée : 0.40 s
$ python clonecompress.py 7z l resultats/texte.txt.7zc
Date                 Taille     CRC32  Nom
-------------------------------------------------------------
2026-10-20 04:38        310768  2bdbb6f1  texte.txt
-------------------------------------------------------------
                        310768  1 entrée(s)
$ python clonecompress.py bzip2 -d -k -f -v -o resultats/_bz.out resultats/texte.txt.bzc
resultats/texte.txt.bzc -> resultats/_bz.out : 49.65 Ko -> 303.48 Ko en 0.18 s
$ python clonecompress.py lzma -d -k -f -v -o resultats/_lz.out resultats/texte.txt.lzc
resultats/texte.txt.lzc -> resultats/_lz.out : 69.87 Ko -> 303.48 Ko en 0.16 s
$ python clonecompress.py 7z x resultats/texte.txt.7zc -o resultats/_7z
Extrait   texte.txt
Extraction terminée dans 'resultats/_7z'.
# Vérification SHA-256 de l'aller-retour : bzip2=OK  lzma=OK  7z=OK
```

3.2 Fichier audio : audio.wav

L'audio PCM bruité compresse peu : le bruit de fond est incompressible sans pertes. Le gain, faible, vient des parties tonales périodiques.

Taille originale : **264 644 o** — bzip2-clone : **237 596 o** (89.8 %) — lzma-clone : **245 576 o** (92.8 %) — 7z-clone : **245 697 o** (92.8 %).

Session de terminal :

```
$ python clonecompress.py bzip2 -k -f -v -n 3 -o resultats/audio.wav.bzc echantillons/audio.wav
echantillons/audio.wav -> resultats/audio.wav.bzc : 258.44 Ko -> 232.03 Ko (89.78 % de l'original, gain 10.22 %)
    en 0.72 s
$ python clonecompress.py lzma -k -f -v -n 6 -o resultats/audio.wav.lzc echantillons/audio.wav
echantillons/audio.wav -> resultats/audio.wav.lzc : 258.44 Ko -> 239.82 Ko (92.79 % de l'original, gain 7.21 %)
    en 0.92 s
$ python clonecompress.py 7z a resultats/audio.wav.7zc echantillons/audio.wav -n 5
Archive resultats/audio.wav.7zc créée : 1 fichier(s), 258.44 Ko -> 239.94 Ko (92.84 % de l'original)
Durée : 0.93 s
$ python clonecompress.py 7z l resultats/audio.wav.7zc
Date                 Taille     CRC32  Nom
-------------------------------------------------------------
2026-10-20 04:38        264644  b35ee8a1  audio.wav
-------------------------------------------------------------
                        264644  1 entrée(s)
$ python clonecompress.py bzip2 -d -k -f -v -o resultats/_bz.out resultats/audio.wav.bzc
resultats/audio.wav.bzc -> resultats/_bz.out : 232.03 Ko -> 258.44 Ko en 0.43 s
$ python clonecompress.py lzma -d -k -f -v -o resultats/_lz.out resultats/audio.wav.lzc
resultats/audio.wav.lzc -> resultats/_lz.out : 239.82 Ko -> 258.44 Ko en 0.56 s
$ python clonecompress.py 7z x resultats/audio.wav.7zc -o resultats/_7z
Extrait   audio.wav
Extraction terminée dans 'resultats/_7z'.
# Vérification SHA-256 de l'aller-retour : bzip2=OK  lzma=OK  7z=OK
```

3.3 Fichier image : image.bmp

L'image BMP (non compressée) contient de larges aplats et dégradés : forte redondance spatiale, que LZ77 + codeur par intervalles exploitent mieux que bzip2.

Taille originale : **230 454 o** — bzip2-clone : **161 513 o** (70.1 %) — lzma-clone : **103 851 o** (45.1 %) — 7z-clone : **103 973 o** (45.1 %).

Session de terminal :

```
$ python clonecompress.py bzip2 -k -f -v -n 3 -o resultats/image.bmp.bzc echantillons/image.bmp
echantillons/image.bmp -> resultats/image.bmp.bzc : 225.05 Ko -> 157.73 Ko (70.08 % de l'original, gain 29.92 %)
    en 0.66 s
$ python clonecompress.py lzma -k -f -v -n 6 -o resultats/image.bmp.lzc echantillons/image.bmp
echantillons/image.bmp -> resultats/image.bmp.lzc : 225.05 Ko -> 101.42 Ko (45.06 % de l'original, gain 54.94 %)
    en 0.50 s
$ python clonecompress.py 7z a resultats/image.bmp.7zc echantillons/image.bmp -n 5
Archive resultats/image.bmp.7zc créée : 1 fichier(s), 225.05 Ko -> 101.54 Ko (45.12 % de l'original)
Durée : 0.50 s
$ python clonecompress.py 7z l resultats/image.bmp.7zc
Date                 Taille     CRC32  Nom
------------------------------------------------------------
2026-10-20 04:38        230454  8af887f2  image.bmp
------------------------------------------------------------
                        230454  1 entrée(s)
$ python clonecompress.py bzip2 -d -k -f -v -o resultats/_bz.out resultats/image.bmp.bzc
resultats/image.bmp.bzc -> resultats/_bz.out : 157.73 Ko -> 225.05 Ko en 0.29 s
$ python clonecompress.py lzma -d -k -f -v -o resultats/_lz.out resultats/image.bmp.lzc
resultats/image.bmp.lzc -> resultats/_lz.out : 101.42 Ko -> 225.05 Ko en 0.30 s
$ python clonecompress.py 7z x resultats/image.bmp.7zc -o resultats/_7z
Extrait   image.bmp
Extraction terminée dans 'resultats/_7z'.
# Vérification SHA-256 de l'aller-retour : bzip2=OK  lzma=OK  7z=OK
```

3.4 Fichier vidéo : video.y4m

La vidéo brute (Y4M, images YUV 4:2:0) est très redondante d'une image à l'autre : compression de plus de 99 %. Un fichier déjà compressé (MP4, H.264) ne se compresserait presque pas.

Taille originale : **576 163 o** — bzip2-clone : **4 641 o** (0.8 %) — lzma-clone : **4 898 o** (0.9 %) — 7z-clone : **5 105 o** (0.9 %).

Session de terminal :

```
$ python clonecompress.py bzip2 -k -f -v -n 3 -o resultats/video.y4m.bzc echantillons/video.y4m
echantillons/video.y4m -> resultats/video.y4m.bzc : 562.66 Ko -> 4.53 Ko (0.81 % de l'original, gain 99.19 %) en
    0.87 s
$ python clonecompress.py lzma -k -f -v -n 6 -o resultats/video.y4m.lzc echantillons/video.y4m
echantillons/video.y4m -> resultats/video.y4m.lzc : 562.66 Ko -> 4.78 Ko (0.85 % de l'original, gain 99.15 %) en
    0.24 s
$ python clonecompress.py 7z a resultats/video.y4m.7zc echantillons/video.y4m -n 5
Archive resultats/video.y4m.7zc créée : 1 fichier(s), 562.66 Ko -> 4.99 Ko (0.89 % de l'original)
Durée : 0.25 s
$ python clonecompress.py 7z l resultats/video.y4m.7zc
Date                 Taille     CRC32  Nom
-------------------------------------------------------------
2026-10-20 04:38        576163  dbeddbbb  video.y4m
-------------------------------------------------------------
                        576163  1 entrée(s)
$ python clonecompress.py bzip2 -d -k -f -v -o resultats/_bz.out resultats/video.y4m.bzc
resultats/video.y4m.bzc -> resultats/_bz.out : 4.53 Ko -> 562.66 Ko en 0.10 s
$ python clonecompress.py lzma -d -k -f -v -o resultats/_lz.out resultats/video.y4m.lzc
resultats/video.y4m.lzc -> resultats/_lz.out : 4.78 Ko -> 562.66 Ko en 0.02 s
$ python clonecompress.py 7z x resultats/video.y4m.7zc -o resultats/_7z
Extrait   video.y4m
Extraction terminée dans 'resultats/_7z'.
# Vérification SHA-256 de l'aller-retour : bzip2=OK  lzma=OK  7z=OK
```

4. Conclusion

Les trois clones compressent et décompressent sans perte les quatre types de fichiers. Les gains sont maximaux sur les données redondantes (texte, image non compressée, vidéo brute) et minimaux sur les données bruitées ou déjà compressées, ce qui est conforme à la théorie de l'information (entropie de Shannon). Les implémentations Python sont plus lentes et un peu moins efficaces que les outils officiels (écrits en C, avec des heuristiques de recherche et de modélisation plus poussées), mais respectent les mêmes principes.